

Open-SDD: Executable Architecture, Zero-Trust Harnessing, and Living Specifications for Enterprise Agentic Software Engineering

Mario Alejandro Ramos* Open-SDD Research & Architecture Initiative

September 2026 (Preprint)

Abstract

As generative artificial intelligence transitions from assistive auto-completion to multi-agent autonomous swarms, enterprise software engineering confronts an acute governance crisis: stochastic model capacity outpaces deterministic verification. Uncontrolled agentic pipelines routinely amplify write throughput while severely degrading system cohesion, multiplying code duplication and accumulating latent architectural debt.

This paper introduces **Open-SDD**, an open, model-agnostic, zero-trust specification-driven reference harness and methodological toolkit. Grounded in the fifth generation of software abstraction where architectural intent becomes an executable control plane (*SpecOps*), Open-SDD formalizes a four-phase closed-loop cycle: *propose* $\rightarrow$ *act* $\rightarrow$ *verify* $\rightarrow$ *learn*. By deprecating fragmented, vendor-tied heuristics and eradicating legacy branded wrappers in favor of unified `sdd-*` protocols and `.sdd/` living trees, Open-SDD introduces four structural innovations: (i) a **Documentary Triad** operating under EARS syntax and persistent Git-native provenance; (ii) an **Adaptive Contextual Decision Engine** (PASS/WARN/ASK/BLOCK) mitigating the empirical *Bypass Conjecture* ($E_{\text{ef}} = E_{\text{nom}}(1 - d(\phi))$) to prevent compliance disabling under false-positive fatigue; (iii) a **Brownfield Reverse-Engineering and Drift Discovery** methodology (*Graph before Grep*) for existing enterprise monoliths; and (iv) native alignment with emergent regulatory frameworks including the EU AI Act (Articles 11–15), NIST AI RMF, and ISO/IEC 42001. We demonstrate how Open-SDD relocates human authority to intent, ethics, and boundary-setting while transforming code into an ephemeral, regenerable commodity.

Keywords: Spec-Driven Development, Agentic Software Engineering, Zero-Trust Architecture, Living Specifications, Brownfield Modernization, Human-in-the-Loop, AI Governance.

1 Introduction: The Inflection Point of 2026

Three years into the widespread deployment of Large Language Model (LLM) coding assistants, empirical data reveals a striking divergence between perceived acceleration and objective delivery stability. Randomized controlled trials by METR demonstrated that experienced engineers operating AI assistants without formal constraints were up to 19% slower on mature codebases despite self-reporting a 20% speed improvement [1]. Longitudinal source-code evaluations across 211 million lines by GitClear revealed a fourfold multiplication in duplicated code blocks and a decline of refactoring below 10% of total change volume [2]. Similarly, DORA empirical telemetry confirmed that for every 25 percentage-point increase in ungoverned AI adoption, delivery stability suffered a 7.2% degradation [3].

*Computer Engineer specialising in Spatial Computing & Agentic Architectures, NTT DATA. Email: mario.ramos@nttdata.com

The diagnostic conclusion is unambiguous: **the bottleneck in enterprise agentic engineering is not generative capability, but the absence of a deterministic control plane surrounding a stochastic process.** When left uninstrumented, autonomous agents create state coupling across interdependent components, lack transactional wave atomicity, and suffer from algorithmic collusion when generating and reviewing code within shared context windows.

To solve this dilemma, we present **Open-SDD**, an enterprise-grade, model-agnostic reference harness that transforms architectural specifications from passive, perishable documentation into executable, auditable, and enforceable reality. Open-SDD repudiates proprietary platform lock-in and vendor prefixes (eradicating all historical **kiro-*** baggage), replacing them with an open standard rooted in Git primitives, EARS syntax, and deterministic validation gates.

2 Theoretical Foundations: The Fifth Generation of Abstraction

As articulated in recent architectural literature from Red Hat’s Office of the CTO [4], software engineering history is defined by progressive abstraction leaps:

1. **1st Generation:** Machine code (direct binary manipulation).
2. **2nd Generation:** Assembly mnemonics (symbolic processor operations).
3. **3rd Generation:** High-level compiled languages (*C*, Fortran, Java) masking platform instructions.
4. **4th Generation:** Managed memory and dynamic runtimes (garbage collection, virtual machines, declarative infrastructure).
5. **5th Generation (SDD):** Declarative natural and structured intent as the primary executable control surface (*Spec-Driven Development*).

In this paradigm shift, code ceases to be the canonical source of truth. As illustrated in Table 1, SDD enacts a fundamental architectural inversion. Implementation code becomes disposable and regenerable on demand—*Ambient Code*—while the specification repository hosted natively in Git governs the runtime invariant.

Table 1: Architectural Inversion: Classical vs. Open-SDD Model (adapted from [4]).

Dimension	Classical Model (Legacy)	Open-SDD Model
Source of Truth	Source code defines behavior	Executable specifications define behavior
Role of Architecture	Advisory (dormant diagrams in wikis)	Enforceable, executable, and binding
Code Lifecycle	Primary long-term enterprise asset	Ambient code: disposable, reproducible
Drift Detection	Forensic post-facto (production bugs)	Pre-runtime prevention via continuous gates
Runtime Nature	Emergent and unpredictable	Architecturally deterministic
Human Oversight	Forensic code review line-by-line	High-level intent, policy, and boundary gating

3 The Core Architecture: The Propose–Act–Verify–Learn Loop

Open-SDD rejects unconstrained "vibe coding" by encapsulating agent workflows in a closed cybernetic loop:

$$\text{Loop} = \langle \text{Propose} \longrightarrow \text{Act} \longrightarrow \text{Verify} \longrightarrow \text{Learn} \rangle \quad (1)$$

The underlying philosophical principle states: *the model proposes, the harness disposes.*

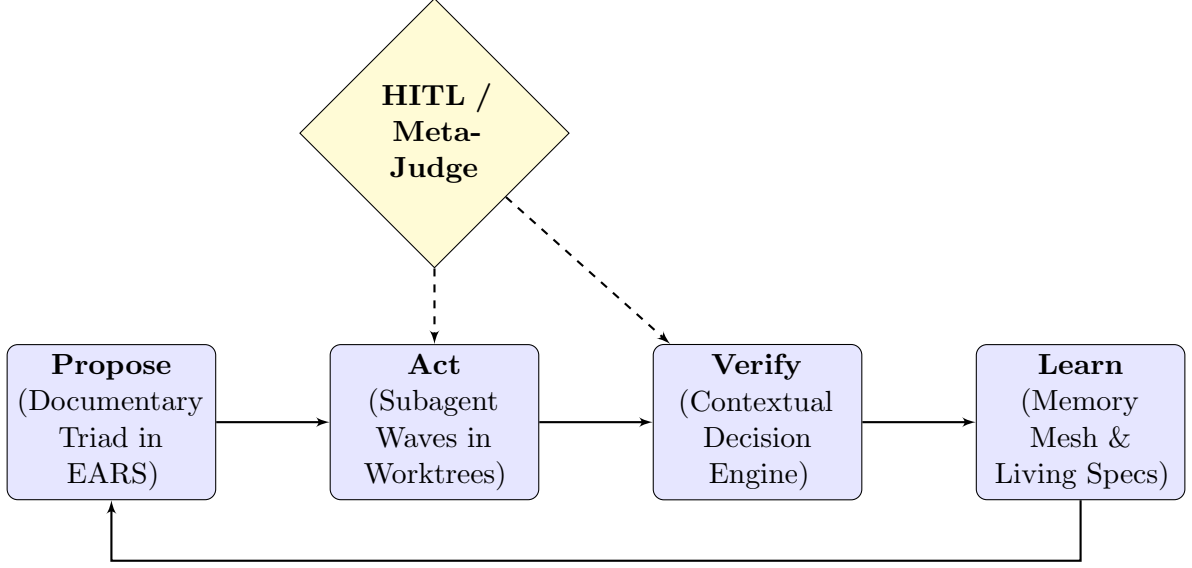


Figure 1: The Open-SDD cybernetic control envelope with Meta-Judge and HITL oversight.

3.1 1. Propose: The Documentary Triad

No generation proceeds without a bounded specification structured into three synchronized artifacts stored in `.sdd/specs/{feature}/:`

- **requirements.md**: Expressed strictly in **EARS** (Easy Approach to Requirements Syntax) [7]. All acceptance criteria must follow ubiquitous, event-driven, state-driven, or unwanted behavior patterns (e.g., *WHEN [event] THEN the system SHALL [action] WHERE [constraint]*).
- **design.md**: Component topologies, schemas, interface contracts, and immutable **Architecture Decision Records (ADRs)**.
- **tasks.md**: Directed Acyclic Graphs (DAG) organizing atomic, test-driven execution waves with explicit dependency annotations (`_Depends:-`, `_Boundary:-`).

3.2 2. Act: Transactional Worktrees and Subagent Roles

To prevent mutable state races, each task executes within an isolated Git worktree on an ephemeral wave branch. Parallel tasks share no dirty filesystem. If a sub-task fails its acceptance criteria, its worktree is discarded; the primary repository state remains untainted by construction. Rollback ceases to be a manual emergency procedure and becomes the automatic abandonment of an unmerged branch.

3.3 3. Verify: The Bypass Conjecture and Contextual Decision Engine

A central theoretical finding of Ramos [5] is that rigid binary gating induces governance self-destruction:

$$E_{\text{ef}}(\phi) = E_{\text{nom}}(\phi) \cdot (1 - d(\phi)) \quad (2)$$

where E_{nom} is the nominal enforcement rate, ϕ is the false-positive rate, and $d(\phi)$ is the human-induced disablement probability. When a harness raises false alarms on harmless representations (e.g., flagging `rm -rf` within a test fixture or cleaning up a mock sandbox), developers predictably export bypass variables (e.g., `BYPASS=1`), driving real security $E_{\text{ef}} \rightarrow 0$.

Open-SDD resolves this through a 6-dimensional **Contextual Decision Engine** (evaluating phase, environment, surface, artifact type, model confidence, and risk score) producing four graduated verdicts:

- **PASS**: Benign representations confirmed in non-executable fixtures or documentation.
- **WARN**: Acceptable process debt recorded in the audit trail without pipeline termination.
- **ASK**: Ambiguous boundary detection requiring Human-in-the-Loop (HITL) approval via single-use, hash-bound digital receipts.
- **BLOCK**: Unconditional halt reserved for confirmed critical execution risks (e.g., unverified destructive shell commands, live credential exfiltration).

3.4 4. Learn: Memory Mesh and Anti-Poisoning Quarantine

Knowledge gained during development is partitioned into a dual-memory system:

- **Session/Temporal Memory** (`.sdd/memory/session/`): Compressed ledgers retaining transient decisions and pruned logs to comply with token budget constraints.
- **Structural Knowledge Graph** (`.sdd/memory/graph/`): A repository-wide index mapping symbols, imports, and AST blast radii (*Graph before Grep*).
- **Anti-Poisoning Quarantine**: Prevents indirect prompt injection by demanding that lessons captured upon gate failures pass a temperature-zero curator screening and cryptographic provenance signature before promotion to permanent steering.

4 Karpathy Behavioral Directives Mechanized

Informal practitioner wisdom, notably synthesized by Andrej Karpathy [6], highlights recurring LLM failure modes: overcomplication, premature abstraction, and scope drift. Open-SDD elevates these behavioral rules into machine-enforced constraints:

Table 2: Mechanization of Karpathy Guidelines in Open-SDD.

Guideline	Open-SDD Mechanism	Operational Invariant
Think Before Coding	Discovery Phase (<code>sdd-discovery</code>)	Prohibits silent assumption; forces clarifying dialogue before spec initialization.
Simplicity First	Hard Gate	Forbids single-use abstractions, unrequested configurability, and excessive diffs.
Surgical Changes	Boundary Gate	Restricts file writes strictly to declared task boundaries; rejects "polishing" untouched code.
Goal-Driven Execution	Verify Gate (<code>sdd-verify</code>)	Rejects task completion without captured exit codes, passing assertions, and fresh stdout evidence.

5 Brownfield Enterprise Modernization: The 4-Step Strategy

Deploying agentic SDD across legacy codebases within hyperscalers (AWS, GCP, Azure) requires avoiding brittle full-rewrite traps. Open-SDD codifies a repeatable four-phase onboarding protocol:

1. **Structural Baseline via Graph Extraction:** The harness scans ASTs and module boundaries to generate a dependency and call graph in `.sdd/memory/graph/`. This bounds the blast radius of any future task before a single grep is executed.
2. **Golden Characterization Locking:** Legacy endpoints are bounded by automated characterization tests capturing existing runtime behaviors and edge-case quirks.
3. **EARS Invariant Extraction:** The core enterprise requirements are codified retroactively into `.sdd/steering/` (`product.md`, `tech.md`, `structure.md`) and feature specs.
4. **Transactional Wave Refactoring:** Incremental additions or refactorings execute in isolated subagent waves. If tests break the golden baseline, the worktree is rolled back cleanly.

6 Auditability, Traceability, and Regulatory Compliance

In heavily regulated environments, an untraced architectural decision is an unauthorized decision. Open-SDD incorporates a dedicated compliance audit command (`sdd-audit`) that compiles machine-readable and PDF reports answering key regulatory mandates:

- **EU AI Act Compliance:** Full traceability of technical documentation (Art. 11), automatic logging of autonomous sessions (Art. 12), and quantified human oversight logs (Art. 14).
- **NIST AI RMF & ISO/IEC 42001:** Continuous risk mapping and verifiable control evidence bound to git commit trailers.
- **Decision Genealogy:** Every deployed function links bi-directionally to its originating EARS requirement ID, ADR justification, executing agent model hash, and verifying human approver.

7 Conclusion

Open-SDD demonstrates that autonomous software development at scale does not require more capable models; it demands disciplined, deterministic harness engineering. By treating specifications as living, versioned source code in Git, mechanizing surgical simplicity, and replacing naive binary barriers with contextual zero-trust decision engines, Open-SDD provides the foundational infrastructure necessary to turn stochastic agentic generation into predictable, enterprise-ready software engineering.

References

- [1] J. Becker, N. Rush, E. Barnes, and D. Rein. Measuring the impact of early-2025 AI on experienced open-source developer productivity. *arXiv preprint arXiv:2507.09089*, 2025.
- [2] GitClear. AI Copilot code quality: 2025 data suggests 4x growth in code clones. *GitClear Research Reports*, 2025.
- [3] Google Cloud DORA. *Accelerate State of DevOps Report 2024–2025: AI Adoption and System Stability*. Google, 2025.
- [4] L. Griffin and R. Carroll. Spec driven development: When architecture becomes executable. *InfoQ Architecture Series*, January 2026.

- [5] M. A. Ramos. SDD-First multi-agent orchestration for enterprise development: A zero-trust reference architecture for governing agentic software engineering at scale. *Preprint DOI: 10.13140/RG.2.2.29185.42087*, August 2026.
- [6] A. Karpathy. Guidelines for mitigating common LLM coding pitfalls. Codified in <https://github.com/multica-ai/andrej-karpathy-skills>, 2025–2026.
- [7] A. Mavin, P. Wilkinson, A. Harwood, and M. Novak. Easy approach to requirements syntax (EARS). In *Proc. 17th IEEE International Requirements Engineering Conference*, 2009.
- [8] L. Sampaio. *AI-Assisted SDD: Spec-Driven Development with Gemini, Claude, and ai-sdd*. Self-Published, 2025.